

PROJECT REPORT
ON
"WEDUCATION" — AN ONLINE EDUCATION PORTAL
(DEVELOPED WITH PYTHON & DJANGO)

**Submitted in partial fulfillment of the requirements
for the award of the
DIPLOMA IN COMPUTER SCIENCE & ENGINEERING**



Name: MIHIR PAUL

Roll No: 2408CSE024

Registration No: 24406060024

Semester: IV (4th Semester)

Under the Guidance of:

CODEZEAL TECHNOLOGY PVT. LTD. Namkum, Ranchi.



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
BEHARAGORA POLYTECHNIC, BAHARAGORA

Recognized By

Jharkhand University of Technology (JUT), Ranchi.

DECLARATION BY THE STUDENT

I hereby declare that this Diploma Project Report entitled "WEDUCATION — AN ONLINE EDUCATION PORTAL" submitted to the Department of Computer Science & Engineering, BEHARAGORA POLYTECHNIC, BAHARAGORA, affiliated to Jharkhand University of Technology (JUT), Ranchi, is an authentic record of our own work carried out under the joint guidance of Mr. Priyaranjan Nayak (H.O.D. of CSE) and my host Internship Company.

The findings, designs, and code integrations presented in this report are original, and no part of this report has been submitted elsewhere for any other degree or diploma.

(Signature of the Candidate)

Name: MIHIR PAUL

BONAFIDE & INTERNSHIP CERTIFICATE

This is to certify that the project report entitled "WEDUCATION" is a bonafide record of the work done by MIHIR PAUL, Regd. No:24406060024 in partial fulfillment of the requirements for the award of Diploma in Computer Science & Engineering from State Board of Technical Education, Jharkhand under Jharkhand University of Technology (JUT), Ranchi, during the academic session 2025-2026.

The project was successfully completed as part of a Web Development training & internship at **CODEZEAL TECHNOLOGY PVT. LTD. Namkum, Ranchi.**

Signature of the Faculty Guide

Signature of the HOD (CSE)

Signature of Company Director

[OFFICIAL COMPANY STAMP]

Place: _____

Date: _____

ACKNOWLEDGEMENT

I express my profound gratitude to Jharkhand University of Technology (JUT), SBTE Ranchi, and the Principal of Beharagora Polytechnic, Baharagora for allowing us to execute this practical application project.

I am deeply indebted to our Head of Department, Mr. Priyaranjan Nayak, and my internal guide for their constant support. I would like to offer special thanks to the CODEZEAL TECHNOLOGY PVT. LTD. Namkum, Ranchi. and our team lead for giving me the platform to work on production-ready Django environments.

Lastly, I want to thank my lab instructors and friends who shared code debugging ideas and supported me through the design, database connectivity, and routing steps.

MIHIR PAUL

Regd. No: 24406060024

4th Sem, Diploma CSE

INDEX

S.No.	Topic
01.	Abstract
02.	Introduction
03.	System Requirement
04.	Technology Used (Python & Django)
05.	SDLC (Software Development Life Cycle)
06.	ER Diagram
07.	Class Diagram
08.	Database Design
09.	Flow Chart
10.	Coding
11.	Screenshots
12.	Testing
13.	Conclusion & Future Scope
14.	Bibliography

ABSTRACT

Weducation is a responsive online education portal developed as a Diploma project using the Django web framework. The portal aims to bridge the communication and learning gap between instructors and students by providing a secure system for course uploads, class materials, and online student tracking. The application uses Python as the programming engine, Django's built-in SQLite database engine for database records, and standard frontend components (HTML5, CSS3, Bootstrap 5) to ensure a completely responsive and user-friendly visual layout.

Core features implemented in Weducation include Django authentication views (separate user flows for Student and Instructor), dynamic course catalog pages, a file-upload engine for assignments, and an interactive feedback section. SQLite handles all the backend structured data models. This project demonstrates modular Django architectures, clean database styling, and secure CRUD (Create, Read, Update, Delete) handling typical of professional enterprise-level web architectures.

INTRODUCTION

Overview of Weducation Portal

In the modern era of computing, digital learning has turned into a necessity. Weducation is a responsive Online Education and content delivery portal engineered with Python's Django framework. It offers structured course sections, resource sharing capabilities, and instructor dashboards. This project allows engineering diploma students to master the Model-View-Template (MVT) architecture of Django, styling logic flows, and database designs.

Profile of the Internship Company / Organization

The development of this project was mentored and supervised under the industrial training curriculum at [Company Name]. [Company Name] is a premium IT consulting and web development firm founded in [Year], and specialized in building modern cloud infrastructures and Python-Django enterprise products. Under the direction of the Company's Director and senior web development developers, the training spanned over a course of 4 to 6 weeks, covering requirements mapping, wireframing, backend styling, and local testing workflows.

Problem Statement

Traditional classroom setups have resource sharing limitations. Existing open-source learning management systems are either overly heavy to host, require expensive subscriptions, or lack clean, simple user interaction panels. The Weducation portal addresses these core limitations by offering a highly secure, lightweight, and completely responsive platform.

Project Objectives

- To design and develop an Online Education platform using Django framework.
- To implement a secure Student & Instructor registration and login system using Django Auth.
- To create dynamic database models for handling Courses, Lessons, Assignments, and

User

Profiles.

- To build a responsive, interactive user interface utilizing HTML5, CSS3, and Bootstrap 5 styles.

Scope

The current scope of the system covers user registration and authentication, course publishing and management, student enrollment, role-based dashboards, and a public-facing informational site (Home, About, Contact). The system is designed to be extended in the future with lesson/video content delivery, payment gateway integration, and seat-limit enforcement, as outlined in the Conclusion and Future Scope section of this report.

SYSTEM REQUIREMENT

Hardware Requirements

Component	Minimum Requirement
Processor	Dual Core 2.0 GHz or above
RAM	4 GB (8 GB recommended)
Hard Disk	1 GB free space (for project, dependencies, and media uploads)
Monitor	1366 x 768 resolution or higher
Input Devices	Keyboard and Mouse

Software Requirements

Component	Specification
Operating System	Windows 10/11, Linux, or macOS
Programming Language	Python 3.10+
Web Framework	Django 6.0
Front End	HTML5, CSS3, Bootstrap 5, JavaScript
Database	SQLite (development), PostgreSQL (production)
Code Editor	Visual Studio Code
Version Control	Git

TECHNOLOGY USED (PYTHON & DJANGO)

Python

Python is a general-purpose, interpreted, high-level programming language known for its readable syntax and extensive standard library. Its object-oriented design and gentle learning curve make it a popular choice for building web applications quickly and reliably. Python's large ecosystem of third-party packages — including Django itself — was a key reason for its selection as the backend language for this project.

- **Interpreted** — Python code is executed line by line at runtime, without a separate compilation step.
- **Object-Oriented** — Python supports classes and objects, enabling modular, reusable code (used extensively for Django's Models, Views, and Forms in this project).
- **Extensive Standard Library** — reduces the need for external dependencies for common tasks.
- **Large Community and Ecosystem** — mature packages exist for nearly every requirement, from authentication to deployment.

Django Framework

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the Model-View-Template (MVT) architectural pattern:

- **Model** — defines the structure of the application's data (e.g., Courses, Enrollment, ContactMessage) and maps directly to database tables through Django's Object-Relational Mapper (ORM).
- **View** — contains the business logic that processes requests and returns responses (e.g., enrolling a student in a course, or restricting the faculty dashboard to staff users).
- **Template** — defines how data is presented to the user through HTML pages rendered dynamically with real data.

Django was chosen for this project because it provides a built-in admin interface, a robust authentication system, an ORM that eliminates most raw SQL, and strong security defaults (CSRF protection, SQL-injection prevention, and clickjacking protection) out of the box.

SDLC (SOFTWARE DEVELOPMENT LIFE CYCLE)

The Software Development Life Cycle (SDLC) is a structured process followed to design, develop, and test high-quality software. It ensures that the final product meets or exceeds requirements while remaining within time and resource constraints. This project followed the classic Waterfall model, appropriate for a project with a clearly defined and relatively fixed set of requirements.

Planning and Requirement Analysis

The core requirement — a platform connecting faculty and students through course listings and enrollment — was identified first. Functional requirements (registration, course management, enrollment) and non-functional requirements (security, usability, deployability) were both considered during this phase.

System Design

Based on the requirements, the database schema (Courses, Enrollment, ContactMessage models) and the overall application architecture were designed. The ER Diagram and Class Diagram in the following sections were produced during this phase.

Implementation (Coding)

The application was implemented in Django, following the MVT pattern. Views were written for registration, login, course browsing, course detail, enrollment, and faculty course management (add/edit/delete), each secured with the appropriate access control decorators.

Testing

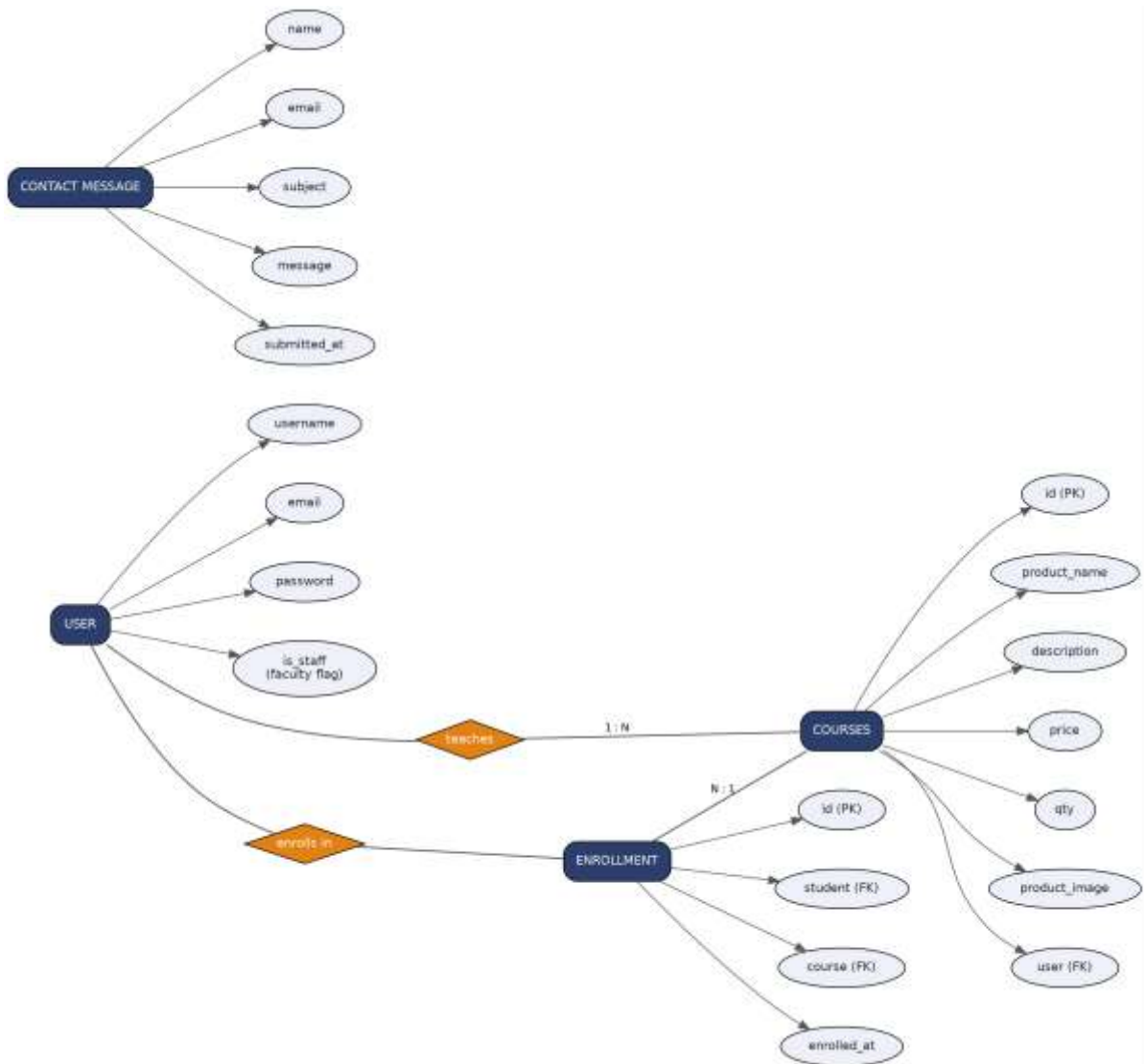
Each feature was tested individually and then as part of complete user journeys — for example, registering a new student, enrolling in a course, and confirming the enrollment appears on the student dashboard. Faculty-only views were tested to confirm that students are correctly blocked from accessing them.

Deployment and Maintenance

The application was configured for production deployment using environment-based settings (SECRET_KEY, DEBUG, ALLOWED_HOSTS), Whitenoise for static file serving, and Gunicorn as the application server, ready to be hosted on platforms such as Render or Railway.

ER DIAGRAM

The Entity-Relationship (ER) diagram below models the core data entities of the Weducation platform and the relationships between them. Rectangles represent entities, ellipses represent their attributes, and diamonds represent relationships between entities.



Entity-Relationship diagram of the Weducation database

Entities

- User — Django's built-in authentication model. A user is treated as Faculty if the `is_staff` flag is set to True, and as a student otherwise.

- Courses — represents a course published by a faculty member, including its name, description, price, quantity, and thumbnail image.
- Enrollment — a join entity linking a Student (User) to a Courses record, representing that the student has enrolled in that course.
- ContactMessage — stores messages submitted through the public Contact Us form.

Relationships

- A User (Faculty) teaches many Courses — a one-to-many (1:N) relationship.
- A User (Student) can enroll in many Courses, and a Course can have many enrolled students — a many-to-many (M:N) relationship, implemented through the Enrollment join table.

CLASS DIAGRAM

The class diagram below represents the Django model classes that make up the application, along with their attributes and the relationships between them, following standard UML notation.

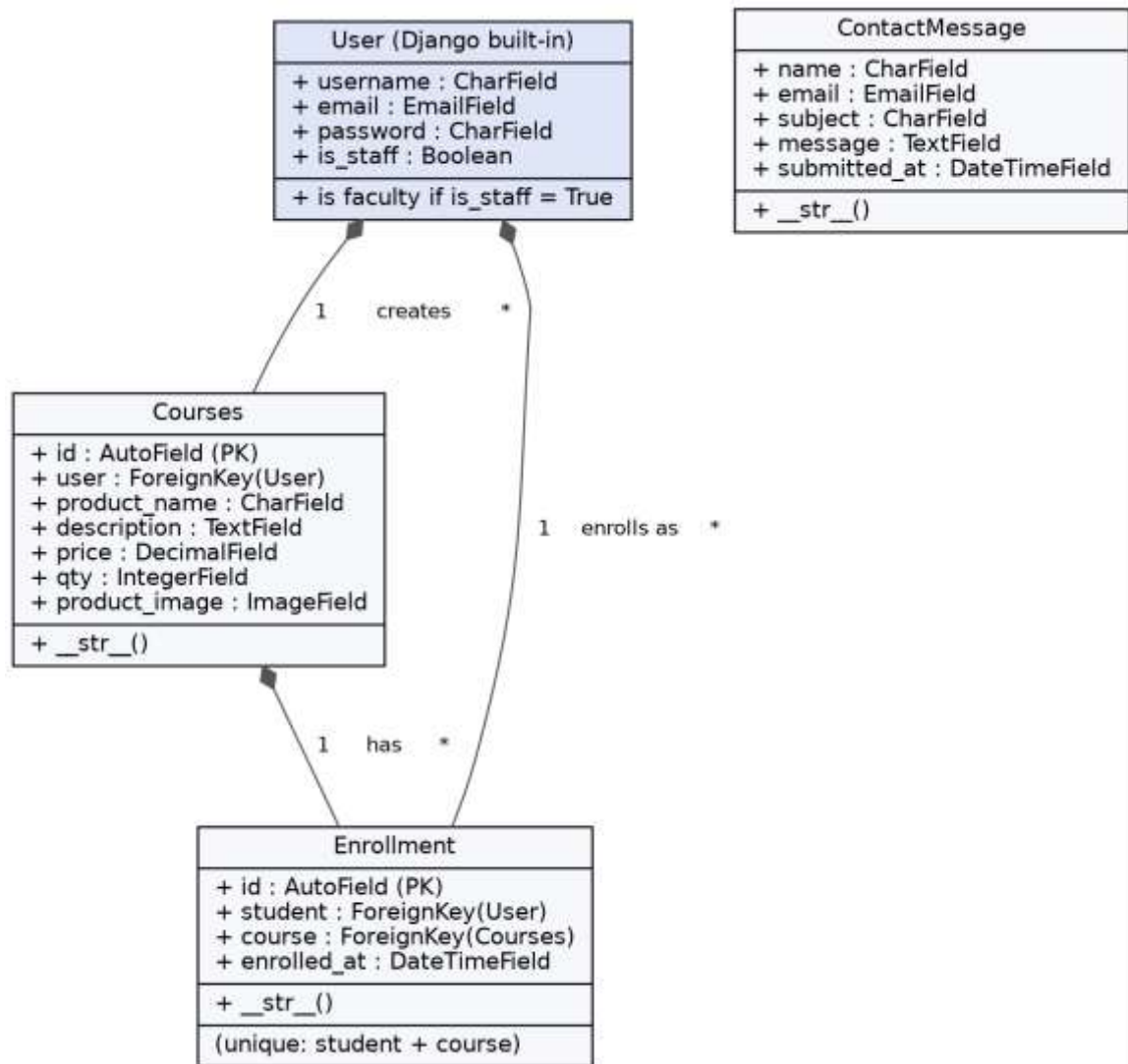


Fig 7.1 — UML class diagram of the Django models

Each class corresponds directly to a Django Model, which Django's ORM maps to a database table. The relationships between **Courses**, **Enrollment**, and **User** are implemented using Django's `ForeignKey` field, which enforces referential integrity at the database level.

DATABASE DESIGN

The following tables describe the schema of each model as implemented in the Django application.

Courses Table

Field	Type	Description
id	AutoField (PK)	Unique identifier for the course
user	ForeignKey (User)	The faculty member who created the course
product_name	CharField(200)	Name/title of the course
description	TextField	Detailed description of the course
price	DecimalField	Price of the course
qty	IntegerField	Seat quantity (reserved for future enforcement)
product_image	ImageField	Course thumbnail image

Enrollment Table

Field	Type	Description
id	AutoField (PK)	Unique identifier for the enrollment record
student	ForeignKey (User)	The student who enrolled
course	ForeignKey (Courses)	The course enrolled in
enrolled_at	DateTimeField	Timestamp of enrollment (auto-set)

ContactMessage Table

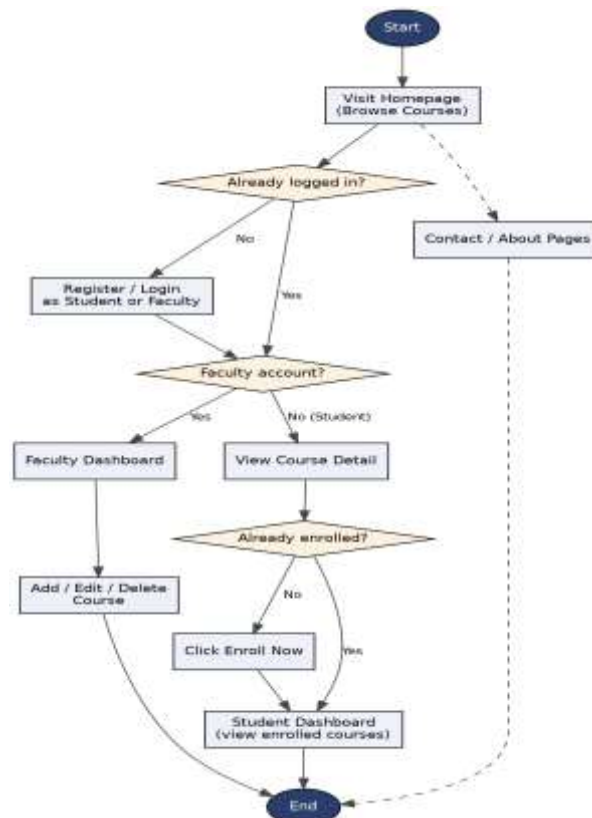
Field	Type	Description
name	CharField(100)	Name of the person submitting the message
email	EmailField	Email address of the sender
subject	CharField(200)	Subject of the inquiry
message	TextField	Body of the message
submitted_at	DateTimeField	Timestamp of submission (auto-set)

FLOW CHART

A flowchart is a diagrammatic representation of the sequence of logical steps in a process. The chart below illustrates the primary user journey through the Weducation platform, covering both the student and faculty paths.

Flowchart Symbols Used

Symbol	Purpose
Oval	Start / End of the process
Rectangle	A process or action step
Diamond	A decision point (Yes / No branch)
Arrow	Shows the direction of process flow



CODING

This section presents key excerpts from the actual Django implementation, showing the Model definitions, a sample View, and the URL routing configuration.

models.py — Courses and Enrollment Models

```
class Courses(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    product_name = models.CharField(max_length=200)
    description = models.TextField(null=True)
    qty = models.IntegerField(default=1)
    product_image = models.ImageField(upload_to='product_images/', blank=True,
null=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
        return self.product_name

class Enrollment(models.Model):
    student = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='enrollments')
    course = models.ForeignKey(Courses, on_delete=models.CASCADE,
related_name='enrollments')
    enrolled_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('student', 'course') # prevents duplicate enrollment
```

— Enrollment and Course Management Views

```
@login_required
def enroll_course(request, course_id):
```

```

course = get_object_or_404(Courses, id=course_id)
if request.method == 'POST':
    Enrollment.objects.get_or_create(student=request.user, course=course)
    messages.success(request, f'Enrolled in {course.product_name}')
    return redirect('course_detail', course_id=course.id)

@login_required
@user_passes_test(is_faculty, login_url='facultylogin')
def add_course(request):
    if request.method == 'POST':
        form = CourseForm(request.POST, request.FILES)
        if form.is_valid():
            course = form.save(commit=False)
            course.user = request.user
            course.save()
            return redirect('facultydashboard')
    else:
        form = CourseForm()
    return render(request, 'pages/course_form.html', {'form': form, 'action':
'Add'})

```

urls.py — URL Routing

```

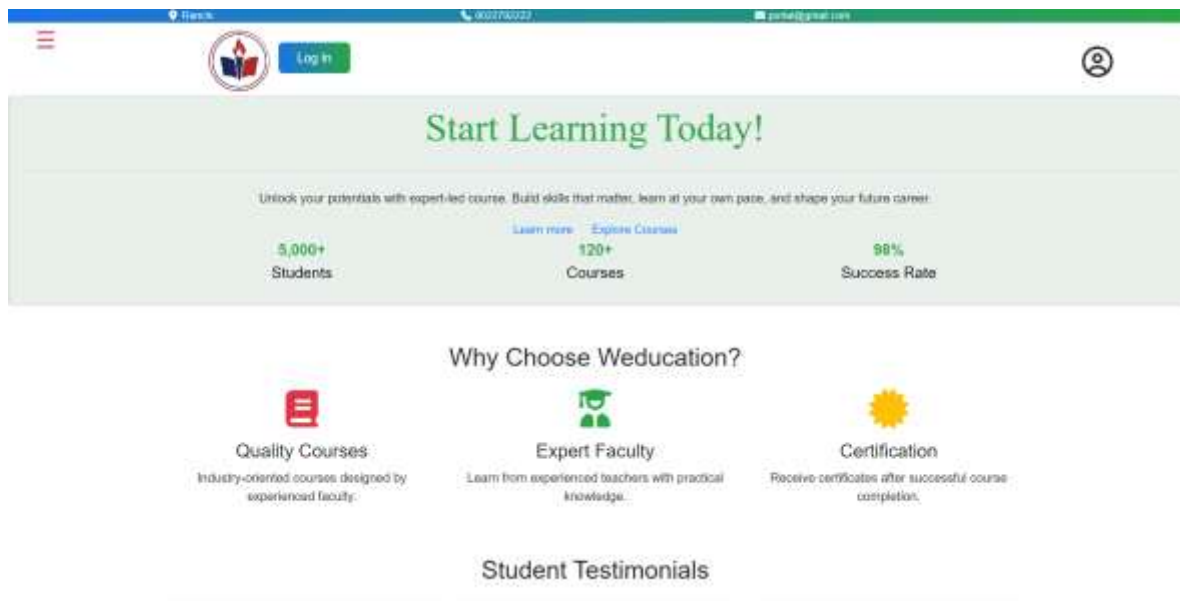
urlpatterns = [
    path('', indexview, name='index'),
    path('about/', about_view, name='about'),
    path('contact/', contact_view, name='contact'),
    path('register/', registerview, name='register'),
    path('studentdashboard/', stdash, name='studentdashboard'),
    path('facultydashboard/', fcdash, name='facultydashboard'),
    path('facultydashboard/add/', add_course, name='add_course'),
    path('course/<int:course_id>', course_detail, name='course_detail'),
    path('course/<int:course_id>/enroll/', enroll_course, name='enroll_course'),]

```

SCREENSHOTS

The following pages display screenshots of the running application. Replace each placeholder box below with an actual screenshot of your deployed or locally running site.

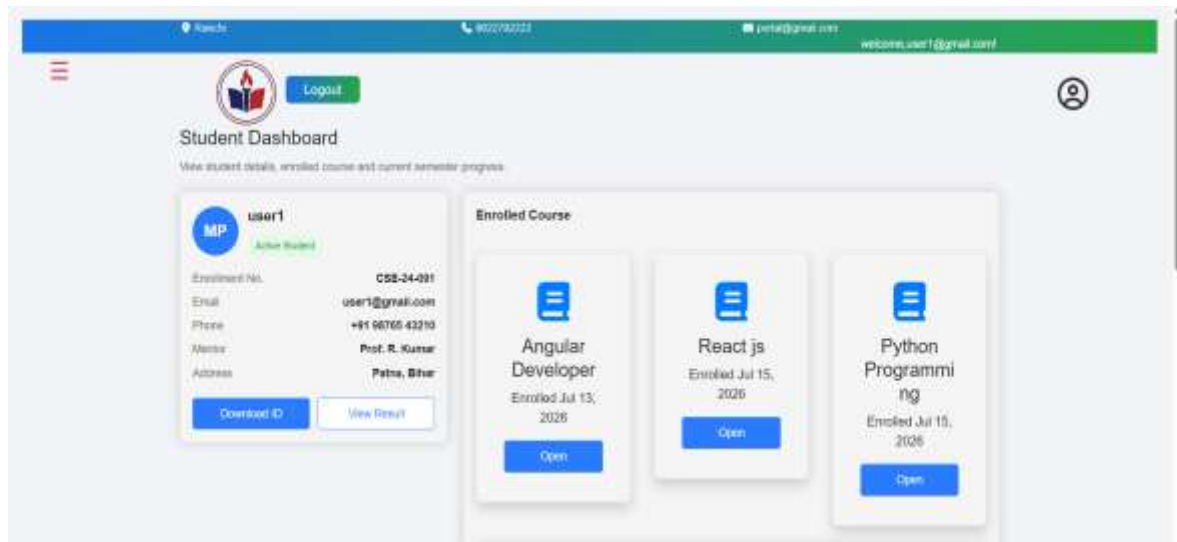
Homepage



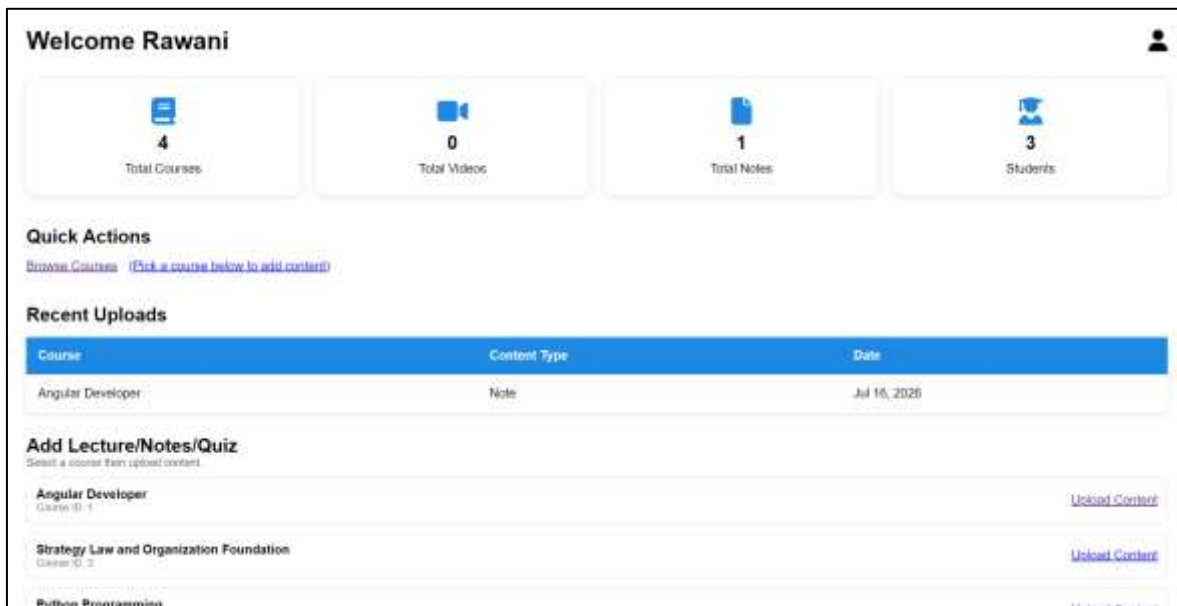
Course Detail Page



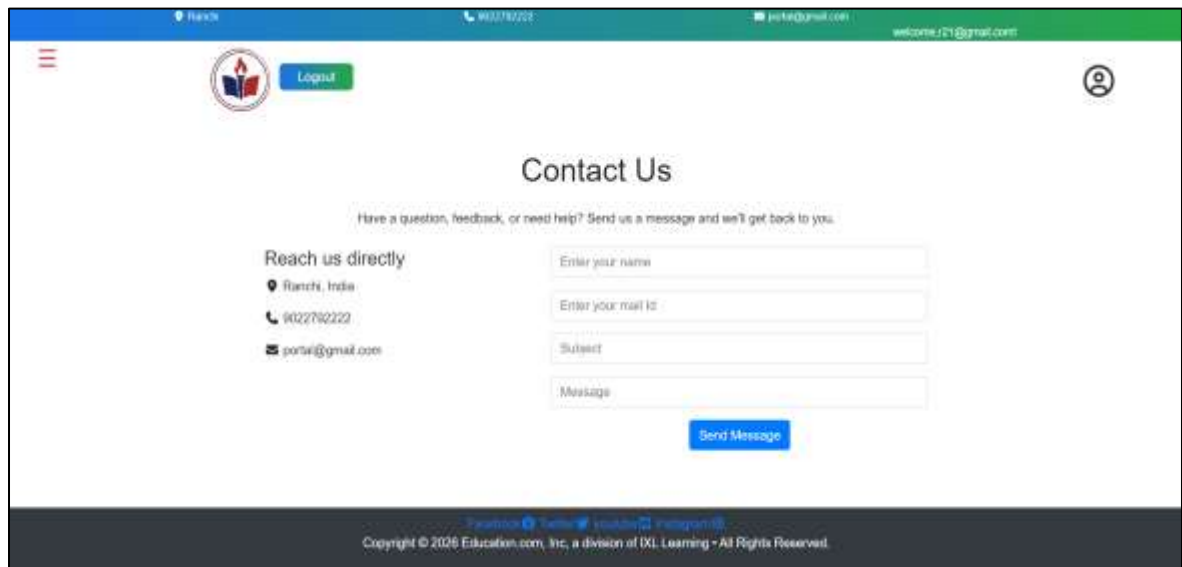
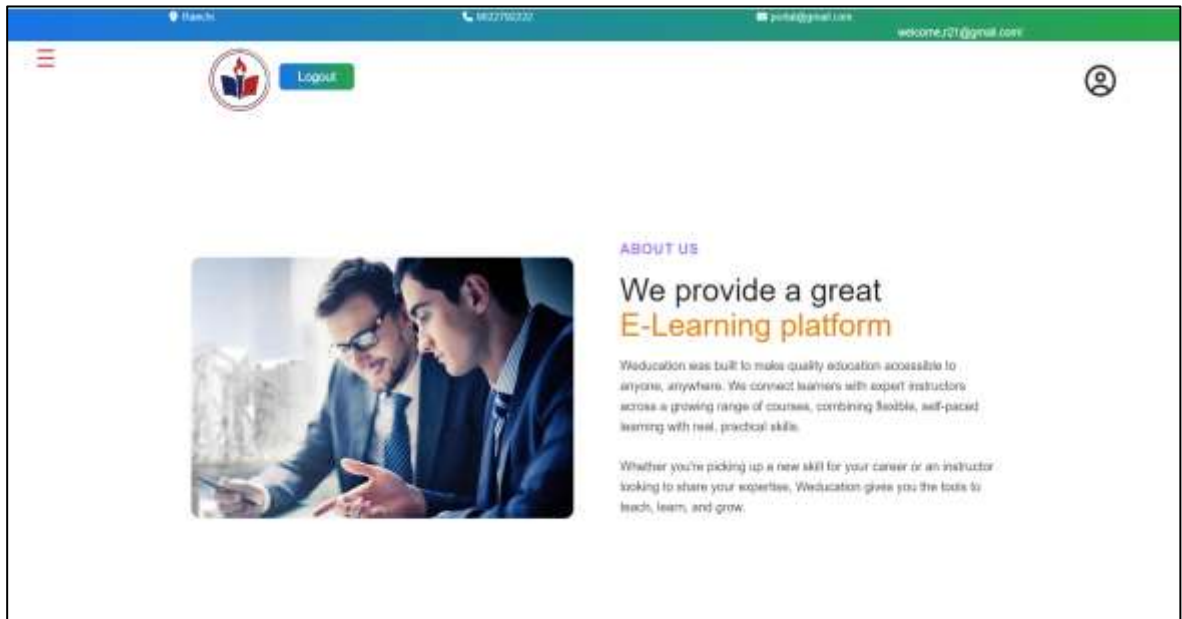
Student Dashboard



Faculty Dashboard



About and Contact Pages



TESTING

The application was tested using Django's test client to simulate real user interactions end-to-end, covering both successful and failure paths.

Test Case	Expected Result	Status
Register with mismatched passwords	Registration rejected, user not created	Pass
Register with valid details	Account created, redirected to login	Pass
Login with correct credentials	Redirected to appropriate dashboard	Pass
Login with incorrect credentials	Login rejected with error message	Pass
Student enrolls in a course	Enrollment created, appears on dashboard	Pass
Student enrolls twice in same course	No duplicate enrollment created	Pass
Faculty adds a new course	Course created and listed on dashboard	Pass
Faculty edits/deletes own course	Change applied successfully	Pass
Student attempts to access faculty dashboard	Access denied, redirected to faculty login	Pass
Faculty attempts to edit another faculty's course	404 Not Found returned	Pass
Contact form submitted with valid data	Message saved, visible in admin panel	Pass
Static files (CSS) served correctly	Stylesheet loads with HTTP 200	Pass

13. CONCLUSION & FUTURE SCOPE

Conclusion

The Weducation project successfully demonstrates a complete, functional online education portal built using Django. It implements the core requirements of a learning management platform: role-based access for students and faculty, course publishing and management, student enrollment, and personalized dashboards. The system has been tested end-to-end and configured for production deployment, following security and configuration best practices appropriate for real-world hosting.

Future Scope

- Lesson Content Delivery — adding a Lesson model so enrolled students can access actual video or text-based course material, rather than enrollment simply marking a course as "joined".
- Seat Limit Enforcement — using the existing qty field on the Courses model to cap enrollments once a course reaches capacity.
- Payment Gateway Integration — enabling paid course purchases through a provider such as Razorpay or Stripe.
- Course Progress Tracking — recording which lessons a student has completed within a course.
- Ratings and Reviews — allowing students to rate and review completed courses.
- Migration to PostgreSQL and cloud media storage for full production scalability.

BIBLIOGRAPHY

- Django Software Foundation — Django Documentation, <https://docs.djangoproject.com/>
- Python Software Foundation — Python 3 Documentation, <https://docs.python.org/3/>
- Bootstrap Team — Bootstrap 5 Documentation, <https://getbootstrap.com/docs/>
- Mozilla Developer Network — HTML, CSS, and JavaScript References, <https://developer.mozilla.org/>
- Whitenoise Documentation, <https://whitenoise.readthedocs.io/>
- Render Documentation — Deploying Django Applications, <https://render.com/docs/>